

The Role of Randomness in Cryptographic Security

"How do Cryptographically Secure Pseudo-Random Number Generators (CSPRNGs) compare to Non-Cryptographically Secure PRNGs in terms of statistical randomness, computational efficiency, and suitability for cryptographic applications?"

Table of Contents

1 Introduction.....	4
2 Background and Theoretical Framework.....	5
2.1 Randomness in Computing and Cryptography	5
2.2 Entropy and Unpredictability.....	6
2.2.1 Shannon’s Entropy.....	6
2.3 Random Number Generators	7
2.3.1 True Random Number Generators (TRNGs).....	7
2.3.2 Pseudo-Random Number Generators (PRNGs).....	8
2.3.3 Cryptographically Secure Pseudo-Random Number Generators (CSPRNGs).....	10
2.4 Statistical Tests for Randomness	11
2.4.1 Chi-Square Test	11
2.4.2 NIST SP 800-22.....	12
2.4.3 Linear Correlation.....	12
3 Methodology	14
3.1 Method Overview	14
3.2 Selection, Implementation, and Output of RNGs	14
3.3 Implementation of Statistical Randomness Tests	16
3.4 Computational Efficiency Tests.....	17
4 Results and Analysis	18
4.1 Entropy and Chi-square Analysis	19
4.2 Linear Correlation Analysis.....	20
4.3 Computational Efficiency Analysis	20

4.4 Suitability for Cryptography	21
5 Conclusion	23
6 Evaluation and Future Work.....	24
6.1 Evaluation of Methodology and Limitations	24
6.2 Suggestions for Future Improvements	25
Works Cited	26
Appendix.....	28
1. Program Code of RNGs and Statistical Tests	28

1 Introduction

In today's online world, randomness is a foundational element of secure communication systems. From encrypting sensitive information to producing usable authentication tokens, the reliability of many cryptographic protocols depends on the reliability of the Random Number Generator (RNG) they use. RNGs are known to be classified under two large categories known as Pseudo-Random Number Generators (PRNGs) and True Random Number Generators (TRNGs).

TRNGs are derived from physical processes that are random, such as thermal noise or radioactive decay, and offers actual randomness but tend to be limited by hardware and cost (Eastlake et al., 2005). As a matter of pragmatics, PRNGs are software-oriented and use deterministic algorithms or mathematical formulas and are widely used across numerous fields, ranging from simulation to games. Cryptographically Secure Pseudo-Random Number Generators (CSPRNGs) are designed to be resistant to cryptanalytic attacks and provide unpredictability, even when some portion of the internal state/output is exposed (Ferguson et al., 2011). Most applications today still rely on PRNGs and CSPRNGs for operation. Thus, the question is: How do Cryptographically Secure Pseudo-Random Number Generators (CSPRNGs) compare to Non-Cryptographically Secure PRNGs in terms of statistical randomness, computational efficiency, and suitability for cryptographic applications?

2 Background and Theoretical Framework

2.1 Randomness in Computing and Cryptography

Randomness refers to the ability to generate values that cannot be feasibly predicted or replicated. In addition, random numbers must be independent from one another (previous outputs will not impact future outputs), and each possible random number generated should be equally probable. Since computers are inherently deterministic, any “random” output must be algorithmically simulated or derived from physical phenomena (such as radioactive decay).

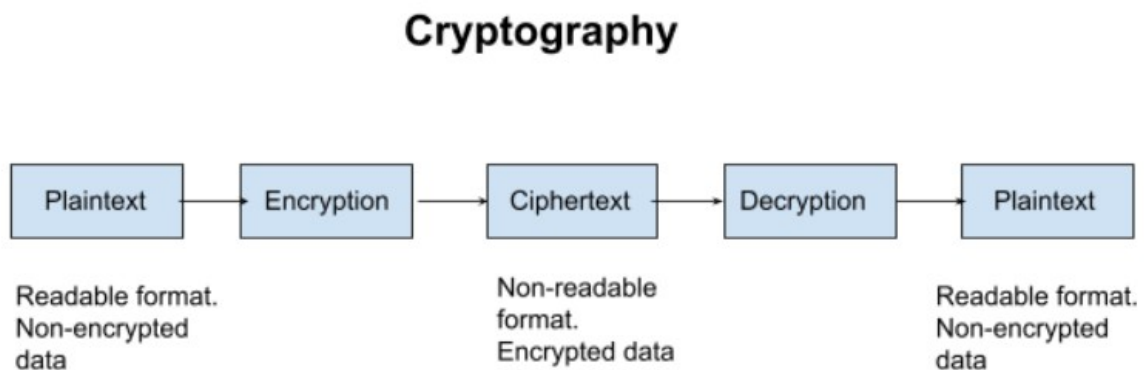


Figure 2.1, A visual representation of cryptography and the process of encryption

Cryptography is the science and practice of securing information and communication by transforming messages/information in ways that are hard to decipher, or to a form where external parties cannot understand (Stallings, W., 2017). In today’s computer-centric world, cryptography is most often associated with scrambling plaintext (ordinary text) into ciphertext (a scrambled

version of plain text done by a process named encryption), then back again into plaintext (in a process named decryption) (Kathleen R., 2024). Figure 2.1 illustrates this process.

These systems rely on secrets (such as encryption keys) that must be impossible for attackers to predict. As such, if the values in cryptographic operations can be guessed (not random), then it would expose it to brute-force attacks and compromise data confidentiality, authenticity, and integrity (BlueGoatCyber, *unknown*) (Yevgeniy Dodis, 2019).

2.2 Entropy and Unpredictability

2.2.1 Shannon's Entropy

Shannon's Entropy (entropy), introduced by Claude E. Shannon in his seminal 1948 paper *A Mathematical Theory of Communication*, is a “mathematical measure of uncertainty or information content in a random variable.” Essentially, the concept measures the degree of unpredictability or ‘surprise’ associated with a set of outcomes. The entropy $H(X)$, measured in bits, of a discrete random variable X with possible outcomes $x_1, x_2, \dots, x_n$ and corresponding probabilities $P(x_1), P(x_2), \dots, P(x_n)$ is given by the formula:

$$H(X) = - \sum_{i=1}^n P(x_i) \log_2 P(x_i)$$

A higher entropy would indicate a more uniform and unpredictable distribution, while lower entropy (possibly due to knowledge of a PRNG seed) indicates predictable outcomes.

A TRNG producing 2-bit values (4 possible outcomes, $n = 2^2$) should have an entropy of 2 bits, and so on. The entropy theory further supports that for a number to be truly random, each possible random number generated should be equally probable ($P(x_1) = P(x_2) = \dots = P(x_n)$). In general, the maximum entropy (H) of a random variable (X) with n equally likely outcomes is:

$$H_{max} = \log_2 n$$

Thus, for a value or RNG of y bits to be considered truly random, it must produce each possible outcome with equal probability, resulting in y bits of entropy.

2.3 Random Number Generators

RNGs are essential for tasks where unpredictability, fairness, or variability is required. Cryptographic functions are no exception; the performance and security of such functions rely on the reliability and unpredictability of the RNG they employ. As stated by the National Institute of Standards and Technology (NIST), a random number generator is “a process that produces a sequence of values that does not exhibit any discernible pattern or regularity” (Barker & Kelsey, 2015). However, all RNGs are not the same. They are broadly categorized into TRNGs or PRNGs

2.3.1 True Random Number Generators (TRNGs)

Although TRNGs are not the main focus of this essay, they still have some relevance in generating seeds, IVs, salts, or nonces for PRNGs and CSPRNGs. TRNGs are a type of RNGs that has high entropy and exhibits truly random properties. TRNGs usually consist of two components: an “unpredictable” source or seed with high entropy, and a *randomness extractor*. To put it simply, a random extractor is capable of taking in weak or imperfect randomness and producing an output

that is very close to truly random, where all outcomes are equally likely (Boaz Barak, Ronen Shaltiel, and Eran Tromer).

However, computationally generating an “unpredictable” source or seed with high entropy can prove to be challenging, as most systems are built on deterministic algorithms. So, physical processes are exploited, using them as a source with high entropy. One example is radioactive decay. Radioactive decay is built from quantum mechanics, meaning the exact moment at which an unstable atomic nucleus will decay cannot be predicted, even in theory (Menezes, van Oorschot, & Vanstone, 1996).

Radioactive decay is then converted into electrical pulses, where the time intervals are digitized and processed. This is where the random extractor comes into play, to remove bias and produce output that is statistically close to a uniform distribution, or equal probability (Killmann & Schindler, 2011). As such, radioactive decay is capable of producing true random numbers with high entropy. However, radioactive entropy sources are not commonly used in RNGs due to practical limitations (such as hardware complexity, safety considerations, etc). As such, pseudo-random number generators (PRNGs) were created

2.3.2 Pseudo-Random Number Generators (PRNGs)

PRNGs are deterministic algorithms that produce sequences of numbers that mimic the statistical properties of randomness, despite being entirely predictable. Each PRNG begins with an initial value known as a seed. PRNGs use a defined mathematical formula to generate a sequence that can be repeated if the seed and formula is known. Because of this deterministic nature, PRNGs

cannot be considered truly random, but can still appear so over short sequences (Menezes, van Oorschot, & Vanstone, 1996).

PRNGs are widely used in applications where speed and the volume of random numbers are prioritized over their randomness or unpredictability, such as simulations and games. Examples of simple yet popular PRNGs are the Linear Congruential Generator (LCG) and the Mersenne Twister (MT19937). The LCG works by using the recurrence relation:

$$X_{n+1} = (aX_n + c) \bmod m$$

Where a , c , and m are constant parameters, and X_0 is the seed. The LCG also uses its previous output ($n-1$) as its seed for the n th output. The Mersenne Twister, meanwhile, generates numbers with a very long period (up to $2^{19937} - 1$) and has better statistical properties than simpler generators (such as LCG), making it the default PRNG in many programming languages such as Python and C++. The Mersenne Twister stores a list of 624 numbers in memory, which it uses to generate each new random number. It also applies a series of bitwise operations (such as XOR and NOT) to improve its randomness/statistical uniformity.

However, due to their determinism, an attacker who knows a part of the sequence, the seed, or the internal state/processes could reverse-engineer everything and predict all future outputs (Kaliski ed., Springer-Verlag, 1997). This violates many cryptographic principles such as forward and backward security, as well as unpredictability (Ferguson et al., 2010). As such, this led to the creation of CSPRNGs, which are explicitly designed to pass these principles and resist cryptanalytic attacks.

2.3.3 Cryptographically Secure Pseudo-Random Number Generators (CSPRNGs)

CSPRNGs are a specialized class of PRNGs designed to resist prediction and withstand cryptanalytic attacks. As such, most cryptographic applications use CSPRNGs instead of TRNGs or PRNGs due to their high performance and cryptographic standards (randomness and unpredictability).

For a PRNG to be considered cryptographically secure, it must pass three requirements:

- Every CSPRNG should pass the next-bit test. That is, given the first k bits of a random sequence, there is no algorithm or method that can predict the $(k+1)th$ bit, unless it's purely by chance.
- Every CSPRNG must contain forward and backward security. Even if an attacker obtains part or all of the CSPRNG's internal state, it should still be impossible to recreate past outputs. Additionally, the generator should also maintain forward security, ensuring that knowledge of its input/past values does not compromise future values.
- A CSPRNG should be indistinguishable from true randomness. This means that no efficient algorithm or process should be able to distinguish between a PRNG/CSPRNG's output and a TRNG's output.

CSPRNGs are also PRNGs and thus need a seed as a starting value. However, CSPRNGs typically use entropy-rich seed sources, typically derived from physical sources or even other TRNGs. CSPRNGs also make use of cryptographic techniques such as block ciphers, hash functions, or stream ciphers to improve security and randomness.

2.4 Statistical Tests for Randomness

We can use a variety of statistical tests that evaluate whether a sequence of numbers exhibits properties expected of a TRNG. Entropy is an example of these tests. A few other tests are:

2.4.1 Chi-Square Test

Chi-square test is a statistical hypothesis test used to compare observed data to expected data. It determines if the generated data deviates significantly from expected values. For n distinct categories and observed counts O_i and expected counts E_i , the chi-square test formula is given by

$$X^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i}$$

Eq (1)

The chi-square test can also determine uniformity in a given sequence, similar to Shannon's entropy. For binary RNGs, the simplest form checks the frequency of 0s and 1s in a given sequence. If a generator outputs 1,000,000 bits, we expect about 500,000 0s and 500,000 1s. The Chi-Square test quantifies how close the actual output is compared to this expectation. This test is also expandable for larger ranges and sizes. The chi-square test is important to determine unpredictability. If the observed data (O_i) is similar to its expected value (E_i), it results in a low chi-square value and a highly predictable sequence.

The test result is then compared to an acceptable range, and if the test statistic exceeds this threshold, the null hypothesis (that the data is uniformly random) is rejected. The chi-square test is used in a lot of statistical test suites to assess whether the output of an RNG exhibits truly random characteristics.

2.4.2 NIST SP 800-22

Developed by the National Institute of Standards and Technology, the NIST 800-22 test suite is one of the most well-known and highly regarded frameworks to test the randomness of sequences generated from RNGs. The suite is composed of 15 sub-tests for different statistical properties of randomness. Thus, this makes NIST highly effective in detecting biases, patterns, correlations, and structural weaknesses that could compromise the reliability or security of the RNG.

2.4.3 Linear Correlation

The linear correlation is a statistical method that measures the degree of dependency between two data points. In this context, linear correlation helps evaluate the independence of consecutive outputs. A truly random sequence should show no correlation between adjacent values, meaning each number is independent of the last. The equation for the linear correlation is:

$$r = \frac{\sum (X_i - \underline{X})(Y_i - \underline{Y})}{\sqrt{\sum (X_i - \underline{X})^2 \sum (Y_i - \underline{Y})^2}}$$

Where:

X_i and Y_i are individual data points,

$\underline{X}$ and $\underline{Y}$ are their respective means,

$$-1 \leq r \leq 1.$$

For a TRNG or CSPRNG, the linear correlation should be approximately zero ($r \approx 0$), indicating no linear dependency between values.

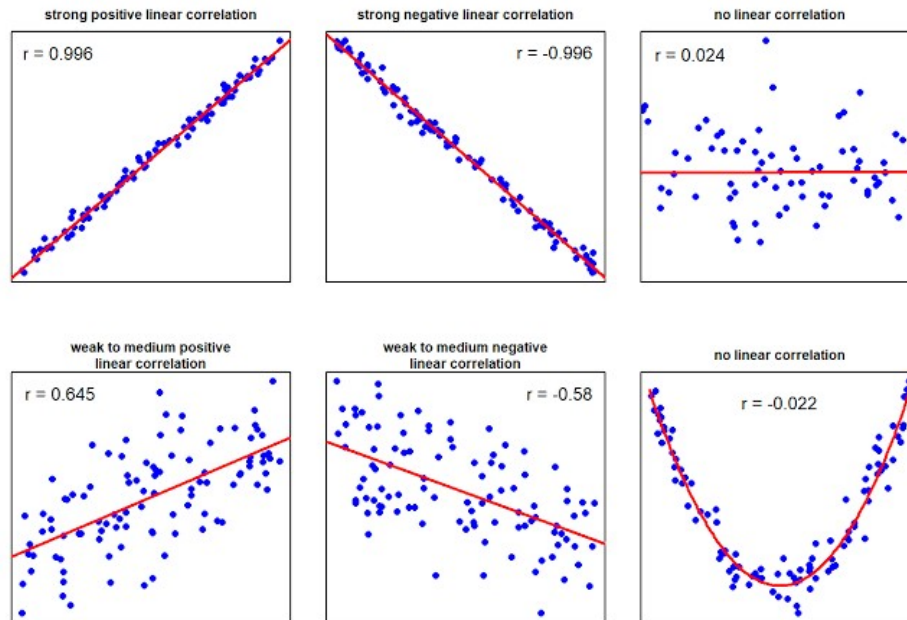


Figure 2.2: Different types of correlation

Figure 2.2 shows a visual representation of linear correlation. A strong positive and negative linear correlation means predictable outputs are highly predictable, just by following the trend line. For example, a strong positive linear correlation would mean that each consecutive output consistently increases. To guess the next output, we would simply increase the previous output linearly. Meanwhile, no linear correlation would mean that outcomes are highly unpredictable (as there is no trend line to follow), which is ideal for an RNG.

3 Methodology

3.1 Method Overview

The methodology is structured around generating large samples of random data (bitstreams of 1 million, 5 million, and 10 million bits) from two PRNGs and CSPRNGs. The PRNGs I will be using are Linear Congruential Generator (LCG) and Mersenne Twister, while I will be using ChaCha20 and Python's *secrets* module for my CSPRNGs. The main programming language to implement these RNGs will be Python. Each output from each generator will then be assessed using the Chi-Square Test, Shannon entropy, Linear Correlation Test, and the NIST SP 800-22 suite to evaluate statistical randomness and unpredictability. Computational efficiency will also be measured by recording the time taken to generate each dataset using Python's *time* library. Qualitative factors, such as algorithm complexity and secrecy, will also be assessed to evaluate their cryptography suitability.

3.2 Selection, Implementation, and Output of RNGs

3.2.1 Selected PRNGs

The first PRNG selected is the Linear Congruential Generator (LCG), which should be the weakest RNG out of the 4. The LCG can be implemented in Python by using its mathematical recurrence relation. Due to the nature of a PRNG, we will not be using a TRNG for its initial seed, as that would be one of the characteristics for a CSPRNG. Instead, we will be using a consistent seed for reproducibility across different runs.

For the second PRNG, we will be using the Mersenne Twister (mt19937), an advanced and popular PRNG known for its respectable statistical properties. We can simply use Python's built-in random module, which uses the Mersenne Twister for its random number generating.

3.2.2 Selected CSPRNGs

The first CSPRNG is ChaCha20, a stream cipher that generates a sequence of pseudo-random bits based on an initial key and nonce. ChaCha20 implements a series of rounds involving additions, rotations, and XOR operations to achieve high randomness and cryptographic security, thus its title as a CSPRNG. ChaCha20's key and nonce are typically sourced from a secure entropy source, like /dev/urandom or even TRNGs. Python's *cryptography* library will be used to implement this CSPRNG.

The second CSPRNG chosen is python's Secrets module, designed specifically for generated cryptographically secure numbers and secrets (such as tokens or passwords). It relies on system (OS)-level entropy sources which ensures the generated values are unpredictable. The same can also be said for its seed generation, where it uses system entropy sources like /dev/urandom. This means we do not have to manually initialize a seed, as the secrets module is part of Python's standard library and can be directly used for generating secure random values.

Note that the implementations for all RNGs can be viewed in the appendix.

3.3 Implementation of Statistical Randomness Tests

After we generate our outputs from each RNG, we have to test whether or not they are applicable for cryptography. These tests are meant to evaluate the statistical randomness, independence, and unpredictability of the sequences generated by the generators. A few of the tests we will be implementing for this investigation are:

- **Chi-Square Test**

- Assess whether the distribution of observed bits (0s and 1s) in a generated sequence matches the expected uniform distribution
- In a truly random sequence, we can expect an equal distribution of 0s and 1s
- The critical value, or the maximum resultant value for the null hypothesis to be accepted, will be calculated in section 4.1 with a 99% confidence level and 255 degrees of freedom. (NIST, 2010)

- **Entropy and Linear Correlation**

- Shannon's Entropy, $H(X)$, will be calculated on 8-bit blocks (bytes) of the output stream to measure average uncertainty. A value close to 8-bits would be ideal, as that would exhibit properties of a TRNG
- Lastly, a scatter plot will be generated by plotting X_i against X_{i+l} to possibly identify short, or even long-range correlations or patterns. A Pearson coefficient of (close to) 0 would be ideal, as it suggest no recurring patterns between outputs.

- **NIST SP 800-22 Testing**

- The NIST test suite (version 2.1.2) will be used to evaluate the randomness of each RNG
- We will assume the null hypothesis (H_0) for all tests is that a sequence is random

- The test will automatically return a pass or fail. A failure means the null hypothesis will be rejected, and the sequence will be deemed non-random with a configured 99% confidence level.

3.4 Computational Efficiency Tests

Cryptography and encryption also require speed in order to be functional, and in fact, that's the exact reason why TRNGs are not so suitable for cryptography. As such, the computational efficiency of each generator must be measured. This is done by timing the duration required to generate the three target bitstream sizes.

To ensure data integrity, all testing will be conducted on the same local machine (Intel(R) Core(TM) Ultra 9 285H running on Windows 11). This will ensure variables like clock speed, memory access, and operating system overhead are consistent across all generator tests. For each generator and each bitstream size, the mean and standard deviation is calculated to reduce bias.

4 Results and Analysis

Each RNG (LCG, Mersenne, ChaCha20, and Secrets) are ran individually and analyzed using Shannon's Entropy, Chi-square, and time which are all programmed manually. This program is executed for all 4 RNGs and all 3 bitstreams (1M, 5M, 10M), with their outputs stored in a .BIN file to be ran through the NIST SP 800-22 test. The following results are obtained:

Generator	Size (M)	NIST test score	Time (s)	Std Dev	Shannon (H)	Chi-square (X^2)	Pearson (r)
LCG	1	5/15	0.015352	0.001550	8.0000	0.11	0.825100
	5	(Fail)	0.084255	0.002991	8.0000	0.03	0.825101
	10		0.266427	0.116752	8.0000	0.01	0.825102
Mersenne	1	14/15	0.049883	0.001683	7.9985	251.10	0.000055
	5	(Pass)	0.259063	0.004808	7.9997	243.43	-0.000100
	10		0.524457	0.005630	7.9998	274.422	0.000155
ChaCha20RNG	1	15/15	0.007327	0.001358	7.9984	277.21	-0.000015
	5	(Pass)	0.047349	0.001170	7.9997	285.48	0.000030
	10		0.099152	0.002022	7.9998	275.04	-0.000087
SecretsRNG	1	15/15	0.010399	0.001088	7.9985	250.84	0.000100
	5	(Pass)	0.058266	0.001278	7.9997	246.98	-0.000050
	10		0.115155	0.003371	7.9998	261.49	0.000211

Table 1: NIST and Statistical results of all 4 RNGs

4.1 Entropy and Chi-square Analysis

Table 1 shows the entropy values for all RNGs are relatively high, very close to the perfect value of 8. This is somewhat expected. Shockingly, LCG scored the highest in the entropy test with a perfect value of 8. Though, this does not necessarily mean that LCG is the standalone best RNG.

First, we must find an acceptable range for the chi-square test. A large X^2 value suggests non-randomness, meaning the generated values deviate significantly from expectation. The ideal value for the chi-square test, in this case, is 255. Thus, for 255 degrees of freedom, the acceptable critical range for a 99% confidence level ($p = 0.01$) is approximately:

$$[204.62, 309.35]$$

The LCG's Chi-Square value of 0.01 is severely outside the acceptable range, proving the sequence is non-uniform and non-random. This suggests that entropy alone is not enough to justify an RNG. Although all 256 bytes appear equally often, entropy fails to detect the non-random ordering or sequence of these bytes, something that X^2 addresses.

The rest of the RNGs passes the chi-square test as all X^2 values fall within the 99% acceptance window. Their high entropy scores also confirm that the byte values are both distributed uniformly and lack any highly probable single outcome.

4.2 Linear Correlation Analysis

The LCG's structural failure is quantified by the Pearson correlation coefficient (r). A r value close to 0 is ideal, while a r close to ± 1 indicates a strong relationship and failure. The Pearson correlation for all 4 RNGs at a bitstream size of 10M are:

Generator	Pearson Correlation (r)	Conclusion
LCG	0.825102	Strong Positive Correlation (Fail)
Mersenne	0.000155	No Correlation (Pass)
ChaCha20RNG	-0.000087	No Correlation (Pass)
SecretsRNG	0.000211	No Correlation (Pass)

The LCG produced an r value of approximately 0.825102, confirming a very strong positive correlation. This violates the independence requirements for cryptography, as the data points fall onto distinct diagonal lines, making the generator inherently deterministic.

Mersenne, ChaCha20RNG, and SecretsRNG have Pearson correlations extremely close to 0. This is great as there is no detectable linear relationship between successive values and aligns with the independence requirement for random sequences.

4.3 Computational Efficiency Analysis

Computational efficiency was measured by the mean time, in seconds, required to generate the largest test size (10M bits)

Generator	10M Bits Time (s)	Speed Factor vs. Mersenne
LCG	0.266427	~ 1.97x Faster
Mersenne	0.524457	1.00x (Baseline)
ChaCha20RNG	0.099152	~ 5.29x Faster
SecretsRNG	0.115155	~ 4.55x Faster

ChaCha20RNG proved to be the most computationally efficient generator, generating 10 million bits in under 0.1 seconds, with Secrets following closely at 0.12 seconds. This demonstrates that a modern CSPRNGs can be significantly faster and suitable for cryptography than all other types tested. Cryptographic security does not necessarily compromise efficiency

The Mersenne Twister proved to be the slowest, at 0.524457 seconds to generate 10M bits. LCG is nearly twice as fast as Mersenne, but its statistical flaws make this speed irrelevant for any serious application such as cryptography

4.4 Suitability for Cryptography

Right away, we can conclude that LCG is not cryptographically secure. It has a low X^2 uniformity, and failed the NIST SP 800-22 test. While the Mersenne Twister (MT) achieved a strong 14/15 Pass on the NIST Suite, excellent X^2 uniformity, and a high Entropy, it is not classified as a CSPRNG. This is because of state recoverability and backward/forward prediction.

As discussed previously, a sequence is only considered cryptographically secure only if an attacker cannot predict the next output, even if they have observed a large portion of the previous outputs. The MT algorithm has a known, large internal state (624 32-bit integers). Crucially, this state is fully observable. Once an attacker collects 624 consecutive 32-bit, they can mathematically reverse-engineer the entire internal state of the generator. With the state known, all future outputs can be predicted perfectly. The MT algorithm is also the slowest of them all, undermining its suitability for cryptography.

In contrast, CSPRNGs like ChaCha20 and Secrets RNG are designed with unpredictability. They incorporate mechanisms like mixing with high-entropy seed material and using hashing or nonces to ensure that the internal state cannot be recovered from the output, even if a large amount of the output is known. This is further proven by a perfect score (15/15) on the NIST Suite, an excellent X^2 uniformity, and a high entropy.

5 Conclusion

In conclusion, the comparison for the 2 PRNGs (LCG and Mersenne Twister (MT)), as well as the 2 CSPRNGs (ChaCha20RNG and SecretsRNG) yielded different results in terms of computational efficiency and statistical/security performance. Out of the 4 RNGs,

- The LCG is definitely unsuitable for any application requiring true randomness. Despite achieving a perfect Shannon Entropy score of 8.0000, its Chi-Square value (0.01), NIST fail, and the high Pearson correlation ($r \approx 0.825$) confirmed that the sequence is highly predictable, and unsuitable for cryptographic applications.
- The Mersenne Twister demonstrated strong statistical properties, with excellent Chi-Square and correlation values and a high pass on the NIST tests. However, the analysis in Section 4.4 confirmed that its predictability and fully recoverable internal state make it unsuitable for cryptographic applications. Furthermore, MT was the slowest generator tested.
- Both CSPRNGs, ChaCha20RNG and SecretsRNG, proved superior in all measurements. They passed all statistical tests, showed no detectable linear correlation, satisfied the requirement of unpredictability, and had a perfect pass on the NIST tests. Additionally, they were also the most efficient, with ChaCha20RNG being the fastest overall (about 5.29x faster than MT).

The CSPRNGs were overwhelmingly superior in speed. ChaCha20RNG and SecretsRNG were approximately 5.29x and 4.55x faster, respectively, than the Mersenne Twister. While the MT

passes statistical tests, it fails the cryptographic requirement of unpredictability due to its fully recoverable internal state. Conversely, both ChaCha20RNG and SecretsRNG meet both the statistical and cryptographic requirements. So, modern CSPRNGs like ChaCha20 offer the highest level of security and speed, and is superior to non-cryptographic PRNGs like the Mersenne Twister and LCG .

6 Evaluation and Future Work

6.1 Evaluation of Methodology and Limitations

- This investigation only considers a small selection of RNGs: one simple PRNG (LCG), one widely used PRNG (Mersenne Twister), as well as two CSPRNGs (ChaCha20, secrets). This excludes other relevant RNGs, such as AES-CTR. Thus, the conclusions cannot be generalized to all real-world RNG implementations.
- The study focused exclusively on Pseudo-Random Number Generators (PRNGs). The comparison lacks a baseline from a TRNG (such as one based on thermal noise or quantum properties/fluctuations)
- While 10 million bits is a substantial volume for general performance comparison, it is relatively small for a rigorous, deep analysis required by the full NIST SP 800-22. They often need gigabytes of data to detect more hidden correlations

6.2 Suggestions for Future Improvements

- Firstly, future work should generate a much larger bitstream size (1GB or 10GB) to be integrated into the full NIST test to uncover more flaws and correlations
- The comparison could also be extended to include one or more hardware-based TRNG to benchmark the performance and randomness gap between hardware (TRNGs) and software (PRNGs) generation
- Finally, the investigation could also be run in multiple programming languages, such as C++ or Java. The differences in standard-library RNG implementations and compiler optimizations could reveal how language design influences both efficiency and randomness quality, which reduces the bias due to programming language.

Works Cited

Barker, Elaine, and John Kelsey. *Recommendation for Random Number Generation Using Deterministic Random Bit Generators (SP 800-90A Revision 1)*. National Institute of Standards and Technology, June 2015. NIST, <https://www.nist.gov/publications/recommendation-random-number-generation-using-deterministic-random-bit-generators-2>

Stallings, William. *Cryptography and Network Security: Principles and Practice*. 7th ed., Pearson, 2017.

National Institute of Standards and Technology. *A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications (SP 800-22 Revision 1a)*. By Andrew Rukhin et al., Apr. 2010 <https://www.nist.gov/publications/statistical-test-suite-random-and-pseudorandom-number-generators-cryptographic>

GeeksforGeeks. “Cryptography and its Types” *GeeksforGeeks*, 2025, <https://www.geeksforgeeks.org/computer-networks/cryptography-and-its-types/>

Eastlake, Donald, et al. *Randomness Requirements for Security (RFC 4086)*. IETF, June 2005, <https://datatracker.ietf.org/doc/html/rfc4086>.

Blue Goat Cyber. “A Guide to Cryptographically Secure Pseudo-Random Number Generators (CSPRNG).” Blue Goat Cyber, <https://bluegoatcyber.com/blog/a-guide-to-cryptographically-secure-pseudo-random-number-generators-csprng/>.

“Caesar Cipher in Cryptography.” GeeksforGeeks, 23 July 2025, <https://www.geeksforgeeks.org/ethical-hacking/caesar-cipher-in-cryptography/>.

Barak, Boaz, Ronen Shaltiel, and Eran Tromer. “True Random Number Generators Secure in a Changing Environment.” International Workshop on Cryptographic Hardware and Embedded Systems, Springer, 2003. PDF, https://link.springer.com/chapter/10.1007/978-3-540-45238-6_14

Schindler, Werner, and Wolfgang Killmann. “Evaluation Criteria for True (Physical) Random Number Generators Used in Cryptographic Applications.”, Springer, 2003, https://link.springer.com/chapter/10.1007/3-540-36400-5_31.

Ferguson, Niels, Bruce Schneier, and Tadayoshi Kohno. Cryptography Engineering: Design Principles and Practical Applications. Wiley, 2010. PDF, http://pub.deadnet.se/Books_on_Tech_Survival_woodworking_foraging_etc/cryptography_engineering_design_principles_and_practical_applications.pdf.

Appendix

1. Program Code of RNGs and Statistical Tests

```
import time

import math

import random

import os

from collections import Counter


# --- 1. Generator Implementations ---


# 1.1 Linear Congruential Generator (LCG)

class LCG:

    def __init__(self, seed=None, a=1103515245, c=12345, m=2**31):

        # Use a consistent seed for reproducibility across different runs/users

        self.state = seed if seed is not None else 1234567

        self.a = a

        self.c = c

        self.m = m
```

```
def next(self):

    # LCG formula:  $X(n+1) = (a * X(n) + c) \bmod m$ 

    self.state = (self.a * self.state + self.c) % self.m

    return self.state
```

```
def generate_byte(self):

    # Return the least significant 8 bits of the 31-bit state

    return self.next() & 0xFF
```

1.2 Mersenne Twister (MT)

```
class MersenneTwister:
```

```
    def __init__(self, seed=None):

        self.seed = seed if seed is not None else 1234567

        self.rng = random.Random(self.seed)
```

```
    def generate_byte(self):

        # Returns a random integer in the range [0, 255]

        return self.rng.randrange(256)
```

1.3 ChaCha20 (CSPRNG) - Cryptographically Secure

```
try:
```

```
    from cryptography.hazmat.primitives.ciphers import Cipher, algorithms
```

```

from cryptography.hazmat.backends import default_backend

class ChaCha20RNG:

    def __init__(self):

        # Key and Nonce setup using a deterministic process for testing consistency

        seed_value = 1234567

        # Key must be 32 bytes (256-bit)

        self.key = seed_value.to_bytes(32, 'big')

        # Nonce must be 16 bytes

        self.nonce = (seed_value + 1).to_bytes(16, 'big')

        self.cipher = Cipher(algorithms.ChaCha20(self.key, self.nonce), mode=None,
backend=default_backend())

        self.encryptor = self.cipher.encryptor()

        self.buffer = b''

    def generate_byte(self):

        if not self.buffer:

            # Generate a new block of keystream (64 bytes)

            self.buffer += self.encryptor.update(b'\x00' * 64)

        byte_value = self.buffer[0]

```

```

        self.buffer = self.buffer[1:]

        return byte_value

except ImportError:

    # If cryptography is not installed, define a placeholder class

    class ChaCha20RNG:

        def __init__(self):

            raise NotImplementedError("ChaCha20RNG requires the 'cryptography' library.")

        def generate_byte(self):

            raise NotImplementedError("ChaCha20RNG requires the 'cryptography' library.")

# 1.4 Python's 'secrets' Module (CSPRNG)

class SecretsRNG:

    def generate_byte(self):

        # Uses os.urandom

        return os.urandom(1)[0]

# --- 2. Statistical Test Functions ---

def shannon_entropy(data):

    """Calculates Shannon Entropy (H) of a sequence of bytes (Ideal max 8.0)."""

    if not data: return 0.0

```

```

byte_counts = Counter(data)

total_bytes = len(data)


entropy = 0.0

for count in byte_counts.values():

    probability = count / total_bytes

    entropy -= probability * math.log2(probability)


return entropy


def chi_square_test(data):

    """Performs the Chi-Square Test for uniformity of byte distribution (Ideal 255.0)."""

    if not data or len(data) < 256: return 0.0


    total_bytes = len(data)

    # Expected count for each byte value if uniform (E)

    expected_count = total_bytes / 256.0


    # Observed counts (O)

    observed_counts = Counter(data)


    chi_square_sum = 0.0

```

```

for i in range(256):

    observed = observed_counts.get(i, 0)

    chi_square_sum += (observed - expected_count)**2 / expected_count


return chi_square_sum

```

--- 3. Execution Framework ---

```

def run_tests_and_measure(generator_instance, size_bits, runs=5):

    """Generates the byte stream, runs tests, and measures generation time.

    Takes size in BITS and converts to BYTES."""

    size_bytes = size_bits // 8


    # --- Efficiency Measurement ---

    times = []


    for _ in range(runs):

        start_time = time.perf_counter()


        byte_sequence = bytes([generator_instance.generate_byte() for _ in range(size_bytes)])


        end_time = time.perf_counter()

```

```

times.append(end_time - start_time)

mean_time = sum(times) / runs

# Calculate standard deviation
if runs > 1:
    variance = sum([(t - mean_time) ** 2 for t in times]) / (runs - 1)
    std_dev = math.sqrt(variance)
else:
    std_dev = 0.0

# Use the last generated sequence for statistical tests
data = byte_sequence

# --- Statistical Tests ---
results = {
    'generator': generator_instance.__class__.__name__,
    'size_bits': size_bits, # Store size in bits
    'mean_time_sec': mean_time,
    'std_dev_sec': std_dev,
    # Run live tests on the generated data
    'shannon_entropy': shannon_entropy(data),
    'chi_square': chi_square_test(data),

```

```

        'raw_data': data
    }

    return results

def save_bitstream_for_external_tests(data, generator_name, size_bits):
    """Saves the raw bytes to a binary file for NIST input."""
    size_mbits = size_bits // 1_000_000
    filename = f"bitstream_{generator_name}_{size_mbits}M_bits.bin"

    os.makedirs('bitstreams', exist_ok=True)
    filepath = os.path.join('bitstreams', filename)

    try:
        with open(filepath, 'wb') as f:
            f.write(data)

        return filepath
    except Exception as e:
        return f"Error saving file: {e}"

# --- 4. Main Execution ---

if __name__ == "__main__":

```

```

# Configuration

BITSTREAM_SIZES_BITS = [1_000_000, 5_000_000, 10_000_000]

NUM_RUNS_FOR_TIME_TEST = 5 # Number of times to run generation for time average


# List of generators to test

generators = {

    "LCG": LCG(),

    "MersenneTwister": MersenneTwister(),

    "ChaCha20RNG": ChaCha20RNG() if 'ChaCha20RNG' in globals() and
ChaCha20RNG.__name__ == 'ChaCha20RNG' else None,

    "SecretsRNG": SecretsRNG()

}


print("--- RNG Statistical Testing Framework ---")

print("Generating and testing 1Mb, 5Mb, and 10Mb data streams.")

print("-" * 70)


all_results = []


# Iterate through all generators

for gen_name, generator in generators.items():

    if generator is None:

        continue

```

```

display_name = gen_name # Use the class name for display

print(f"\n[Testing Generator: {display_name}]")

# Iterate through all required bitstream sizes
for size in BITSTREAM_SIZES_BITS:

    size_mbits = size // 1_000_000

    print(f"> Running test for {size_mbits} Million bits...")

    try:

        # Run the tests and timing

        results = run_tests_and_measure(generator, size, NUM_RUNS_FOR_TIME_TEST)

        results['generator'] = display_name

        all_results.append(results)

    # Save the raw data for external NIST/Correlation tests

    filepath = save_bitstream_for_external_tests(results['raw_data'], gen_name, size)

    # Print summary

    print(f"  - Mean Time: {results['mean_time_sec']:.6f}s (Std Dev:
{results['std_dev_sec']:.6f}s)")

    print(f"  - Shannon Entropy (Ideal 8.0): {results['shannon_entropy']:.4f}")

```

```

    print(f"    - Chi-Square (Ideal 255.0): {results['chi_square']:.2f}")

    print(f"    - Raw data saved to: {filepath}")


except NotImplementedError as e:

    print(f"    [Error] Skipping {gen_name}: {e}")

except Exception as e:

    print(f"    [CRITICAL ERROR] Failed to test {gen_name} at size {size} bits: {e}")


print("\n" + "=" * 70)

print("TESTING COMPLETE")

print("=" * 70)


print("\n--- Summary Table of Performance and Statistical Indicators ---")

print("-----")

")

print("{:<20} {:<10} {:<15} {:<15} {:<15} {:<15}".format(

    "Generator", "Size (Mb)", "Time (s)", "Std Dev (s)", "Shannon H", "Chi-Square"

))

print("-----")

")


for r in all_results:

    # Use size_bits stored in results

```

```

size_mb_output = r['size_bits'] // 1_000_000

if r['mean_time_sec'] > 0:

    print("{:<20} {:<10} {:<15.6f} {:<15.6f} {:<15.4f} {:<15.2f}".format(

        r['generator'],

        size_mb_output,

        r['mean_time_sec'],

        r['std_dev_sec'],

        r['shannon_entropy'],

        r['chi_square']

    ))

else:

    print(f"{r['generator']:<20} {size_mb_output:<10} {'--- FAILED ---':<15} {'--- FAILED ---':<15} {'--- FAILED ---':<15} {'--- FAILED ---':<15}")

    print("-----")

")

print("\n* Raw binary files have been saved to the 'bitstreams' folder.")

print("Critical Chi-Square range : 99% confidence, df=255")

```